

Propuesta de stack tecnológico y entorno de desarrollo

Estado	Borrador actualizado para revisión del equipo
Fecha	25 de septiembre de 2026
Equipo	Equipo 1 - Lucas Ottonello, Ezequiel Marcenal y Cristian Reyes
Alcance	Entrega de análisis y diseño del 4 de octubre de 2026

La propuesta adopta un stack alineado con los requerimientos obligatorios del laboratorio y prioriza herramientas que el equipo pueda desarrollar, probar y explicar dentro del plazo disponible. La solución se implementará como un monolito modular en .NET 10, acompañado por un worker independiente, aplicaciones web y los servicios de soporte exigidos.

1 Criterios de selección

- Cumplimiento directo de las tecnologías y capacidades exigidas por la letra.
- Curva de aprendizaje razonable para un equipo de tres integrantes.
- Entorno local reproducible mediante Docker Compose.
- Separación entre dominio, casos de uso e infraestructura.
- Servicios reemplazables mediante abstracciones, sin acoplar la lógica de negocio.
- Despliegue administrado para reducir trabajo operativo y concentrar el esfuerzo en la aplicación.

2 Stack tecnológico acordado

Área	Tecnología	Uso y justificación
Plataforma	.NET 10 y C#	Base común de la API, aplicaciones web y worker. .NET 10 es obligatorio.
Entorno de desarrollo	Visual Studio 2026 Community	IDE principal para editar, ejecutar, depurar y probar la solución.
Backoffice	ASP.NET Core Razor Pages	Interfaz administrativa del operador, exigida expresamente por la letra.
Portal y seguimiento	Blazor WebAssembly	Portal del comercio y seguimiento público ejecutados en el navegador.
Aplicación del repartidor	Blazor WebAssembly PWA	Alternativa permitida para equipos de tres, instalable y preparada para operación sin conexión.
API	ASP.NET Core Minimal APIs	Endpoints organizados por módulo y caso de uso, con handlers delgados.
Validación	Validación incorporada de .NET 10	Data Annotations, validación automática y respuestas ProblemDetails, sin FluentValidation.
Persistencia	PostgreSQL, EF Core y Npgsql	Base relacional compartida, DbContext por módulo y migraciones controladas.

Área	Tecnología	Uso y justificación
Autenticación	ASP.NET Core Identity y cookies	Usuarios, roles y claims con cookies HttpOnly y Secure; no se usarán JWT inicialmente.
Caché	Valkey	Caché distribuida mediante IDistributedCache para seguimiento público y configuración tarifaria.
Mensajería	RabbitMQ	Comunicación asíncrona entre la API y el worker, con reintentos y cola de fallidos.
Tiempo real	ASP.NET Core SignalR	Actualización del tablero operativo y posiciones de la flota, exigida por la letra.
Observabilidad	Serilog, OpenTelemetry y Aspire Dashboard	Logs estructurados, métricas, trazas y visualización centralizada.
Pruebas	xUnit, WebApplicationFactory y Testcontainers	Pruebas unitarias, de integración, aislamiento y arquitectura. La biblioteca de arquitectura se confirmará al implementar.
Contenedores	Docker Desktop y Docker Compose	Entorno local reproducible con todos los servicios de soporte.
Integración continua	GitHub Actions	Compilación y pruebas obligatorias en pull requests.
Despliegue	DigitalOcean App Platform	Despliegue automático desde main para la API, web, PWA y worker.
Infraestructura como código	Terraform	Creación y reconstrucción reproducible de los recursos de DigitalOcean.

3 Decisiones principales

3.1 Arquitectura y organización

La solución mantendrá la arquitectura de monolito modular definida en el ADR-001. Cada módulo tendrá un proyecto principal con carpetas Domain, Application e Infrastructure, y un proyecto Contracts separado para los mensajes que otros módulos pueden consumir. Los casos de uso se organizarán como vertical slices.

```
API -> Endpoint -> Command o Query -> Handler -> Dominio
                                     -> Puertos de infraestructura
Infrastructure -> EF Core, Valkey, RabbitMQ y servicios externos
```

El dominio no dependerá de ASP.NET Core, Entity Framework Core, Valkey, RabbitMQ ni servicios externos. Los módulos no compartirán entidades internas ni accederán al DbContext de otro módulo.

3.2 Presentación web

El backoffice se desarrollará con Razor Pages. El portal del comercio y el seguimiento público utilizarán Blazor WebAssembly. La aplicación del repartidor será una PWA Blazor WebAssembly. Este modo evita depender de una conexión permanente para renderizar la interfaz y permite implementar la operación offline exigida para el trabajo en calle.

SignalR se utilizará únicamente para actualizaciones en tiempo real. No se utilizará Blazor Server ni el modo automático, con el fin de mantener un solo modelo de ejecución para las interfaces Blazor.

3.3 API y validación

Los endpoints se implementarán con Minimal APIs y se ubicarán junto al caso de uso correspondiente. Cada endpoint traducirá la petición HTTP a un command o query, invocará al handler y convertirá el resultado en una respuesta HTTP. No contendrá reglas de negocio ni acceso directo a datos.

La validación de formato y datos obligatorios se realizará con las capacidades incorporadas en .NET 10. Las reglas del negocio permanecerán en el dominio o en el handler. Los errores HTTP se representarán de forma consistente mediante ProblemDetails.

3.4 Identidad y seguridad

ASP.NET Core Identity administrará usuarios, contraseñas, roles, claims, bloqueos y recuperación de cuenta. La sesión se mantendrá con cookies seguras, configuradas como HttpOnly y Secure. Las aplicaciones y la API se publicarán bajo un mismo dominio lógico para reducir la complejidad de autenticación. El TenantId y, cuando corresponda, el ComercioId se resolverán del contexto autenticado y nunca se confiarán a partir de valores enviados libremente por el cliente.

3.5 Persistencia y multitenancy

PostgreSQL será la base principal. Entity Framework Core se conectará mediante Npgsql. La instancia será compartida, pero cada módulo será dueño de sus tablas y utilizará su propio DbContext. El aislamiento por fila se aplicará con OperatorId y, cuando corresponda, ComercioId, conforme al ADR-002.

Se utilizarán migraciones controladas y pruebas automatizadas para impedir filtraciones entre inquilinos. Las entidades susceptibles de modificaciones simultáneas incorporarán control de concurrencia optimista.

3.6 Caché y mensajería

Valkey se utilizará exclusivamente como caché distribuida. La aplicación accederá mediante IDistributedCache para evitar que los casos de uso dependan directamente del proveedor. Los primeros casos serán el seguimiento público y la configuración tarifaria, con políticas distintas de expiración e invalidación. Se registrarán métricas de aciertos y fallos.

RabbitMQ conectará la API con el worker independiente. La publicación se coordinará con la persistencia mediante Outbox. Los consumidores serán idempotentes e incluirán reintentos con espera creciente y una cola de mensajes fallidos. La biblioteca cliente concreta se elegirá al implementar; RabbitMQ es la decisión arquitectónica confirmada.

3.7 Observabilidad

Serilog producirá logs estructurados. OpenTelemetry generará métricas y trazas distribuidas con identificadores de correlación. Aspire Dashboard recibirá la telemetría y mostrará latencias, errores, profundidad de cola, sincronizaciones pendientes, avisos fallidos y métricas de caché. No se adoptará el orquestador completo de .NET Aspire: solamente su panel de observabilidad.

3.8 Estrategia de pruebas

xUnit será el framework único. Se escribirán pruebas unitarias para reglas de dominio y handlers, pruebas de integración con WebApplicationFactory y PostgreSQL real mediante Testcontainers, pruebas de aislamiento multitenant y al menos una prueba automatizada de arquitectura. No se incorporarán inicialmente Playwright ni una biblioteca de mocks.

4 Entorno de desarrollo

4.1 Herramientas requeridas

- Visual Studio 2026 Community con las cargas de trabajo ASP.NET y desarrollo web y herramientas de contenedores.
- .NET 10 SDK fijado mediante global.json.
- Docker Desktop con Docker Compose.
- Git y acceso al repositorio de GitHub.

- Un navegador basado en Chromium para probar la PWA, el service worker y el modo sin conexión.
- Terraform para planificar y aplicar la infraestructura de DigitalOcean.

4.2 Servicios locales

Docker Compose levantará PostgreSQL, Valkey, RabbitMQ con su interfaz administrativa y Aspire Dashboard. Las aplicaciones .NET podrán ejecutarse desde Visual Studio durante el desarrollo o como contenedores al validar el entorno completo.

El entorno local utilizará los mismos protocolos que producción, aunque los servicios administrados de DigitalOcean se contratarán solamente cuando sea necesario desplegar.

4.3 Configuración y secretos

- appsettings.json contendrá valores no sensibles y configuraciones predeterminadas.
- Los secretos locales se almacenarán con dotnet user-secrets o en un archivo .env excluido de Git.
- El repositorio incluirá .env.example sin contraseñas reales.
- App Platform suministrará secretos y cadenas de conexión mediante variables protegidas.
- Las versiones de paquetes se centralizarán y las imágenes Docker utilizarán etiquetas explícitas.

4.4 Puesta en marcha local

1. Clonar el repositorio y copiar .env.example como .env.
2. Ejecutar docker compose up -d para iniciar los servicios de soporte.
3. Restaurar y compilar la solución con dotnet restore y dotnet build.
4. Aplicar las migraciones controladas de cada módulo.
5. Ejecutar la API, las aplicaciones web y el worker desde Visual Studio o con dotnet run.
6. Ejecutar dotnet test antes de publicar cambios.

5 Organización inicial del repositorio

```
src/
  Logistica.Api/
  Logistica.Backoffice/
  Logistica.PortalComercio/
  Logistica.SeguimientoPublico/
  Logistica.Repartidor.Pwa/
  Logistica.Worker/
  Modules/
    Envios/
      Logistica.Modules.Envios/
        Domain/
        Application/
        Infraestructure/
      Logistica.Modules.Envios.Contracts/
    Administracion/ Planificacion/ Ejecucion/ Seguimiento/ Deposito/
tests/
  Logistica.UnitTests/
  Logistica.IntegrationTests/
  Logistica.ArchitectureTests/
infra/
  terraform/
docs/
docker-compose.yml
.env.example
global.json
Directory.Packages.props
```

La estructura definitiva podrá ajustarse al crear la solución, sin alterar la regla acordada: cada módulo conserva internamente Domain, Application e Infraestructure y publica solamente su proyecto Contracts.

6 Integración y despliegue

```
Rama de trabajo -> Pull request -> GitHub Actions
GitHub Actions -> restore -> build -> test
Merge aprobado a main -> DigitalOcean App Platform -> despliegue automático
Terraform -> creación, actualización y destrucción de la infraestructura
```

La rama main permanecerá protegida y solamente recibirá cambios mediante pull requests con verificaciones exitosas. App Platform observará main y desplegará la API, las interfaces web y el worker. Terraform definirá los recursos de DigitalOcean, sus variables por ambiente y las operaciones reproducibles de plan, aplicación y destrucción.

App Platform administrará el enrutamiento público y HTTPS, por lo que no se incorporará Caddy. El equipo de tres integrantes está eximido del requisito de ejecutar dos instancias balanceadas de la API.

7 Decisiones operativas pendientes

- Alojamiento definitivo de RabbitMQ en producción: servicio administrado o instancia separada.
- Contratación de DigitalOcean Managed Valkey y PostgreSQL según el costo disponible.
- Despliegue y protección del Aspire Dashboard en el ambiente remoto.
- Biblioteca cliente de RabbitMQ y biblioteca de pruebas de arquitectura.
- Almacenamiento de fotografías, firmas y otros archivos de evidencia.

Estas decisiones no impiden presentar el stack ni comenzar el desarrollo. Se cerrarán antes de implementar o desplegar el componente correspondiente.

8 Decisiones que requieren ADR posterior

- Formalización del modo de renderizado Blazor WebAssembly y sus consecuencias.
- Política de resolución de conflictos de sincronización de la PWA.
- Casos de uso almacenados en caché y reglas de expiración e invalidación.
- Garantía de consistencia entre persistencia y publicación de eventos mediante Outbox.

9 Referencias

- Enunciado del laboratorio, secciones 6.1 a 6.15 y 8.1 a 8.3.
- Microsoft Learn. ASP.NET Core, Blazor WebAssembly, Identity, Minimal APIs y pruebas de integración.
- Npgsql Documentation. Entity Framework Core Provider for PostgreSQL.
- RabbitMQ Documentation. Work Queues y Dead Letter Exchanges.
- DigitalOcean Documentation. App Platform, Managed Databases y Terraform Provider.
- OpenTelemetry y .NET Aspire Documentation.